

p0908r0

Offsetof for Pointers to Members

Published Proposal, 11 February 2018

This version:

<https://wg21.link/P0908r0>

Author:

[Eddie Kohler](#) (Harvard)

Audience:

EWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

The `offsetof` macro should support pointers to members.

Table of Contents

- 1 **Questions**
- 2 **Proposed Wording**
- 3 **Example online discussions of the issue**

The `offsetof` macro, inherited from C and applicable to standard-layout classes (and, conditionally, other classes) in C++, calculates the layout offset of a member within a class. `offsetof` is useful for calculating an object pointer given a pointer to one of its members:

```

struct link {
    ...
};

struct container {
    link l;
};

container* container_from_link(link* x) {
    // x is known to be the .l part of some container
    uintptr_t x_address = reinterpret_cast<uintptr_t>(x);
    size_t l_offset = offsetof(container, l);
    return reinterpret_cast<container*>(x_address - l_offset);
}

```

This pattern is used in several implementations of intrusive containers, such as Linux kernel linked lists (`struct list_head`).

Unfortunately, although `offsetof` works for some unusual member-designators, it does not work for pointers to members. This won't compile:

```

template <typename Container, typename Link, Link (Container::* member)>
Container* generic_container_from_link(Link* x) {
    uintptr_t x_address = reinterpret_cast<uintptr_t>(x);
    size_t link_offset = offsetof(Container, member); // error!
    return reinterpret_cast<Container*>(x_address - link_offset);
}

```

Programmers currently compute pointer-to-member offsets using `nullptr` casts (i.e., the incorrect folk implementation of `offsetof`, which invokes undefined behavior), or by jumping through other hoops:

```
template <typename Container, typename Link, Link (Container::* member)>
Container* generic_container_from_link(Link* x) {
    ...
    alignas(Container) char container_space[sizeof(Container)] = {};
    Container* fake_container = reinterpret_cast<Container*>(container_space)
    size_t link_offset = reinterpret_cast<uintptr_t>(&(fake_container->*member
        - reinterpret_cast<uintptr_t>(fake_container));
    ...
}
```

`offsetof` with pointer-to-member member-designators should simply work. Modern compilers implement `offsetof` using an extension (`__builtin_offsetof` in GCC and LLVM), so implementation need not require library changes. To avoid ambiguity, we propose this syntax:

```
size_t link_offset = offsetof(Container, .*member);
```

§ 1. Questions

Must a pointer-to-member expression in an `offsetof` member-designator be a constant expression (such as a template argument)? The C standard requires that “the expression `&(t.member-designator)` evaluates to an address constant,” which might make this code illegal:

```
struct container {
    char array[200];
};

int index = /* dynamic value */;
size_t offset = offsetof(container, array[index]); // questionable
```

But since several current compilers accept dynamic array indexes, the proposed wording allows any pointer to member.

§ 2. Proposed Wording

In Sizes, alignments, and offsets [**support.types.layout**], modify the first sentence of §1 as follows:

The macro `offsetof (type, member-designator)` has the same semantics as the corresponding macro in the C standard library header `<stddef.h>`, but accepts a restricted set of `type` arguments and a superset of `member-designator` arguments in this International Standard.

Add this paragraph after §1:

An `offsetof member-designator` may contain pointer-to-member expressions as well as `member-designators` acceptable in C. A `member-designator` may begin with a prefix `._` or `.*` operator (e.g., `offsetof (type, ._member_name)` or `offsetof (type, .*pointer_to_member)`). If the prefix operator is omitted, `._` is assumed.

§ 3. Example online discussions of the issue

- [\[LLVMdev\] Evaluation of offsetof\(\) macro](#)
- [Working around offsetof limitations in C++](#)